Module 15 - Python Regex Pattern Matching

Table of Contents

- [ ] 1. What is Regex?
- [ ] 2. The `re` Module - Four Essential Functions
- [ ] 3. Character Classes
- [ ] 4. Quantifiers
- [ ] 5. Anchors
- [ ] 6. Groups and Capturing
- [ ] 7. Real-World Security Patterns
- [ ] 8. Summary

---

1. What is Regex?

Analogy: Imagine you receive 50,000 firewall log lines. You need to find every line that contains an IP address in the `10.0.0.x` range, followed by a `DENY` action, on port `22`. You could read them one by one. Or you could give Python a **template** — a description of what the line looks like — and let it scan every line in milliseconds. That template is a **regular expression**.

A regex is a sequence of characters that defines a **search pattern**. It answers questions like:

- Does this string look like a valid IP address?
- Does this log line contain a CVE ID?
- What are all the email addresses in this file?
- Extract the timestamp, source IP, and action from this log entry.

Why regex matters in cybersecurity

Task	Regex use
Log analysis	Extract timestamps, IPs, actions, status codes from raw logs
Threat detection	Find suspicious patterns (CVE IDs, known exploit strings)
Input validation	Check that an IP address or email address is correctly formatted
SIEM rules	Many SIEM platforms (Splunk, Elastic) use regex for alert conditions

Task	Regex use
Incident response	Search across thousands of log files for an attacker's IP

Regex: love it or hate it

Regex has a reputation for being hard to read. This pattern:

```
^(\d{1,3}\.){3}\d{1,3}$
```

...matches an IPv4 address. It looks intimidating. By the end of this notebook, you'll be able to read and write patterns like this without thinking twice.

The key insight: **every regex is just a combination of a small set of building blocks**. Learn the blocks, and any pattern becomes readable.

In [ ]:
```
import re   # the standard library module for regular expressions - no install need

# First example: does this string contain a CVE ID?
log_entry = "[ALERT] Exploit attempt detected - references CVE-2024-1234 on port 44

# re.search() scans the string and returns a Match object if found, None if not
match = re.search(r"CVE-\d{4}-\d+", log_entry)

if match:
    print(f"CVE found: {match.group()}")
else:
    print("No CVE reference found")
```

---

2. The `re` Module - Four Essential Functions

The `re` module gives you four functions you'll use constantly. All take `(pattern, string)` as arguments.

Function	What it does	Returns
`re.match(pattern, string)`	Checks only at the **start** of the string	Match object or `None`
`re.search(pattern, string)`	Scans the **whole** string for the first match	Match object or `None`
`re.findall(pattern, string)`	Finds **all** non-overlapping matches	List of strings
`re.sub(pattern, replacement, string)`	**Replaces** matches with a new string	New string

The raw string prefix `r""`

Always write regex patterns as **raw strings** with the `r` prefix: `r"pattern"`. This prevents Python from interpreting backslashes (`\d` , `\s` , `\n`) before the regex engine sees them.

```
# BAD  - Python interprets \d before regex sees it
re.search("\d+", text)

# GOOD - raw string: backslash goes directly to the regex engine
re.search(r"\d+", text)
```

In [ ]:
```
import re

log_line = "2024-01-15 08:30:42 DENY 203.0.113.5:4444 -> 10.0.0.1:22"

# --- re.match() - only checks from the START of the string ---
# Useful when you know the string MUST start with a specific pattern
m = re.match(r"\d{4}-\d{2}-\d{2}", log_line)  # looks for a date at position 0
print("match():", m.group() if m else None)     # 2024-01-15

m = re.match(r"DENY", log_line)                 # "DENY" is not at position 0
print("match() from middle:", m)               # None - match only checks the start
```

In [ ]:
```
import re

log_line = "2024-01-15 08:30:42 DENY 203.0.113.5:4444 -> 10.0.0.1:22"

# --- re.search() - scans the WHOLE string for the first match ---
# This is what you'll use most often for log parsing
m = re.search(r"DENY|ALLOW", log_line)          # find action anywhere in the line
print("search():", m.group() if m else None)    # DENY

# Match object methods
m = re.search(r"\d+\.\d+\.\d+\.\d+", log_line)  # find the first IP address
if m:
    print(f"Found: {m.group()}")          # 203.0.113.5
    print(f"At position: {m.start()}-{m.end()}")  # where in the string
```

In [ ]:
```
import re

log_line = "2024-01-15 08:30:42 DENY 203.0.113.5:4444 -> 10.0.0.1:22"

# --- re.findall() - returns ALL matches as a list ---
# Perfect when you need every occurrence, not just the first
all_ips = re.findall(r"\d+\.\d+\.\d+\.\d+", log_line)
print("findall() IPs:", all_ips)    # ['203.0.113.5', '10.0.0.1']

# Find all port numbers (the number after a colon)
ports = re.findall(r":(\d+)", log_line)
print("findall() ports:", ports)    # ['4444', '22']
```

In [ ]:
```
import re
```

```
# --- re.sub() - REPLACE matches with a new string ---
# Essential for anonymising logs before sharing, or redacting sensitive data
raw_log = "2024-01-15 DENY 203.0.113.5 -> 10.0.0.1 user=j.smith password=hunter2"

# Redact the password value before storing or sharing the log
redacted = re.sub(r"password=\S+", "password=[REDACTED]", raw_log)
print(redacted)

# Anonymise all external IPs (replace with a placeholder)
# 203.x.x.x are public IPs in the documentation range
anonymised = re.sub(r"203\.0\.113\.\d+", "[EXTERNAL_IP]", raw_log)
print(anonymised)
```

`re.compile()` - pre-compiling a pattern

When you use the same pattern many times (e.g. scanning 50,000 log lines), compile it first. This is faster because Python parses the pattern once instead of on every call.

In [ ]:
```
import re

log_lines = [
    "2024-01-15 08:30:42 DENY 203.0.113.5:4444 -> 10.0.0.1:22",
    "2024-01-15 08:30:45 ALLOW 192.168.1.10:54321 -> 10.0.0.1:443",
    "2024-01-15 08:30:47 DENY 198.51.100.99:1337 -> 10.0.0.2:3389",
    "2024-01-15 08:30:50 ALLOW 192.168.1.20:60001 -> 10.0.0.1:80",
]

# Compile once, use many times
deny_pattern = re.compile(r"DENY")

denied = [line for line in log_lines if deny_pattern.search(line)]
print(f"Denied connections: {len(denied)}")
for line in denied:
    print(" ", line)
```

Practice - Section 2

Write: Using `re.findall()`, extract all CVE IDs from the vulnerability report below. A CVE ID always has the format `CVE-YYYY-NNNN` where YYYY is a 4-digit year and NNNN is one or more digits.

In [ ]:
```
import re

vuln_report = """
Scan complete. Found 3 critical issues on host 10.0.0.55.
CVE-2024-1234 (CVSS 9.8): Remote code execution in Apache 2.4.
CVE-2023-44487 (CVSS 7.5): HTTP/2 Rapid Reset Attack.
See also advisory ref CVE-2022-0001 for background context.
Patch status: CVE-2024-1234 unpatched, CVE-2023-44487 patched.
"""
```

```
# Your code here - extract all CVE IDs
# Expected: ['CVE-2024-1234', 'CVE-2023-44487', 'CVE-2022-0001', 'CVE-2024-1234', '
```

▶ 💡 **SOLUTION**

Fix: The function below tries to redact all IP addresses from a log line, but the pattern is wrong — it also matches version numbers like `2.4.1`. Fix the pattern so it only matches IP addresses (four groups of 1–3 digits separated by dots).

In [ ]:
```
import re

def redact_ips(log_line: str) -> str:
    # BUG: this pattern matches ANY sequence of digits-dot-digits
    # It will wrongly redact version numbers like "Apache 2.4.1"
    return re.sub(r"\d+\.\d+", "[IP]", log_line)


test_line = "DENY 203.0.113.5 -> 10.0.0.1 using Apache 2.4.1"
print(redact_ips(test_line))
# Should print: DENY [IP] -> [IP] using Apache 2.4.1
# But currently prints something wrong - fix the pattern
```

▶ 💡 **SOLUTION**

---

3. Character Classes

Analogy: A character class is like a security policy category: "any digit", "any uppercase letter", "any alphanumeric character". Instead of spelling out every possibility, you name the category.

Shorthand classes (most common)

Shorthand	Matches	Inverse (NOT)	Example match
`\d`	Any digit `[0-9]`	`\D`	`9`, `4`, `0`
`\w`	Any word character `[a-zA-Z0-9_]`	`\W`	`a`, `Z`, `3`, `_`
`\s`	Any whitespace (space, tab, newline)	`\S`	` `, `\t`, `\n`
`.`	Any character except newline	—	`a`, `5`, `-`, `@`

Custom classes with `[]`

Use square brackets to define your own set of allowed characters:

Pattern	Matches
`[abc]`	Exactly `a`, `b`, or `c`
`[a-z]`	Any lowercase letter
`[A-Z]`	Any uppercase letter
`[0-9]`	Same as `\d`
`[^abc]`	Anything EXCEPT `a`, `b`, or `c`
`[a-zA-Z0-9]`	Any alphanumeric character

In [ ]:
```
import re

log_entry = "2024-01-15T08:30:42Z CRITICAL CVE-2024-1234 src=203.0.113.5 dst=10.0.0

# \d - extract all numeric sequences
numbers = re.findall(r"\d+", log_entry)
print("All numbers:", numbers)

# \w+ - extract all word tokens (letters, digits, underscores)
words = re.findall(r"\w+", log_entry)
print("Word tokens:", words)

# \S+ - extract all non-whitespace chunks (includes punctuation)
chunks = re.findall(r"\S+", log_entry)
print("Non-space chunks:", chunks)
```

In [ ]:
```
import re

# Custom character classes in security context
auth_log = "Failed password for user j.smith from 10.0.0.99 port 22 ssh2"

# Username pattern: lowercase letters, digits, dots, underscores, hyphens
username = re.search(r"for user ([a-z][a-z0-9._-]*)", auth_log)
if username:
    print("Username:", username.group(1))     # j.smith

# Severity labels: uppercase letters only, between 3 and 8 chars
alert = "Severity: HIGH - system compromise detected"
severity = re.search(r"[A-Z]{3,8}", alert)
if severity:
    print("Severity:", severity.group())      # HIGH

# [^...] - match anything except specific characters
# Extract the value after '=' in key=value pairs, stopping at spaces
config_line = "action=DENY src=203.0.113.5 dst=10.0.0.1"
values = re.findall(r"=([^\s]+)", config_line)
print("Values:", values)    # ['DENY', '203.0.113.5', '10.0.0.1']
```

Practice - Section 3

Write: Using character classes, extract all **key=value pairs** from the syslog line below as a list of `(key, value)` tuples. Keys are word characters only; values can be anything up to the next space.

In [ ]:
```
import re

syslog_line = (
    "Jan 15 08:30:42 fw01 kernel: "
    "action=DENY proto=TCP src=203.0.113.5 dst=10.0.0.1 dpt=22 len=44"
)

# Your code here - extract all key=value pairs
# Expected: [('action', 'DENY'), ('proto', 'TCP'), ('src', '203.0.113.5'),
#            ('dst', '10.0.0.1'), ('dpt', '22'), ('len', '44')]
```

▶ 💡 **SOLUTION**

---

4. Quantifiers

Analogy: In a security policy, you might say *"a password must have at least 8 characters"*, or *"a CVE year is exactly 4 digits"*, or *"the word CRITICAL may or may not be present"*. Quantifiers are how regex expresses these quantities.

Quantifier	Meaning	Example	Matches
`*`	0 or more	`\d*`	`""` , `"1"` , `"99"` , `"443"`
`+`	1 or more	`\d+`	`"1"` , `"99"` , `"443"` (not `""`)
`?`	0 or 1 (optional)	`https?`	`"http"` or `"https"`
`{n}`	Exactly n times	`\d{4}`	`"2024"` (exactly 4 digits)
`{n,}`	At least n times	`\d{3,}`	`"100"` , `"1234"` , `"99999"`
`{n,m}`	Between n and m	`\d{1,3}`	`"1"` , `"10"` , `"255"`

Greedy vs lazy

By default, quantifiers are **greedy** — they match as much as possible.
Add `?` after the quantifier to make it **lazy** — match as little as possible.

```
# Greedy: .+ matches as much as possible
re.search(r"<.+>", "<b>CRITICAL</b>")  # matches "<b>CRITICAL</b>" (whole
thing)

# Lazy: .+? matches as little as possible
re.search(r"<.+?>", "<b>CRITICAL</b>")  # matches "<b>" (just the first
tag)
```

In [ ]:

```
import re

# Exact counts: CVE year is exactly 4 digits, sequence is 1 or more
cve_pattern = r"CVE-\d{4}-\d+"
test_strings = [
    "CVE-2024-1234",      # valid
    "CVE-24-1234",        # invalid - year is only 2 digits
    "CVE-2024-0",         # valid - sequence number can be 1 digit
    "CVE-2024-999999",    # valid - sequence can be many digits
    "not-a-cve",          # no match
]

for s in test_strings:
    m = re.search(cve_pattern, s)
    print(f"{s:25} -> {m.group() if m else 'NO MATCH'}")
```

In [ ]:
```
import re

# ? for optional parts: https? matches both http and https
urls = [
    "http://internal.corp/login",
    "https://secure.corp/admin",
    "ftp://old-server.corp/files",
]

http_pattern = re.compile(r"https?://\S+")  # s? means the 's' is optional
for url in urls:
    m = http_pattern.search(url)
    print(f"{url:40} -> {m.group() if m else 'NO MATCH'}")
```

In [ ]:
```
import re

# Greedy vs lazy - important when parsing structured log fields
log_with_quotes = 'action="DENY" reason="port scan detected" analyst="j.smith"'

# Greedy: .+ grabs everything between the FIRST " and the LAST "
greedy = re.findall(r'"(.+)"', log_with_quotes)
print("Greedy:", greedy)   # one big match spanning across multiple fields

# Lazy: .+? stops at the FIRST closing quote it finds
lazy = re.findall(r'"(.+?)"', log_with_quotes)
print("Lazy:  ", lazy)     # three separate values - what we actually want
```

Practice - Section 4

Write: Using quantifiers, write a pattern that validates a **port number** string. A valid port number is a whole number between 1 and 65535. The pattern should match the full string (no extra characters before or after). Test it against the list provided.

In [ ]:
```
import re

# Your code here - write a pattern that matches valid port numbers only
# Hint: a port number is 1 to 5 digits. Use anchors (covered in Section 5)
# to prevent partial matches.

test_ports = ["22", "443", "65535", "0", "65536", "8080", "abc", "80abc", "3 389"]
# For now, match strings that are 1-5 digits and check the list
# (We'll add full 1-65535 range validation using anchors in Section 5)
```

▶ 💡 **SOLUTION**

---

5. Anchors

Analogy: Anchors pin your pattern to a specific location in the string — like requiring a form field to start with a capital letter and end with a full stop. Without anchors, a pattern for

`\d+` would match the `22` inside `Apache/2.2.27` — not what you want when validating a port number.

Anchor	Meaning	Use case
`^`	Start of string (or line with `re.MULTILINE`)	Validate that a string starts correctly
`$`	End of string (or line with `re.MULTILINE`)	Validate that a string ends correctly
`\b`	Word boundary (between `\w` and `\W`)	Prevent partial-word matches
`\B`	NOT a word boundary	Match inside a word

`^` and `$` for full-string validation

In [ ]:
```
import re

# Without anchors: matches a valid-looking IP even inside garbage strings
no_anchor = re.compile(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}")
print(no_anchor.search("not_an_ip=garbage10.0.0.1more_garbage"))  # Match! (wrong)

# With anchors: the ENTIRE string must be an IP address
with_anchor = re.compile(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$")
print(with_anchor.match("10.0.0.1"))              # Match
print(with_anchor.match("garbage10.0.0.1more"))   # None - anchors reject this
```

In [ ]:
```
import re

# \b - word boundary: prevents matching partial words
log = "DENY denied pre-DENY post"

# Without \b: matches DENY inside 'denied' too
print(re.findall(r"DENY", log))        # ['DENY', 'DENY', 'DENY'] - matches inside

# With \b: only matches the whole word DENY
print(re.findall(r"\bDENY\b", log))    # ['DENY', 'DENY'] - 'denied' excluded
```

In [ ]:
```
import re

# re.MULTILINE flag: ^ and $ match at the start/end of EACH LINE, not just the whol
firewall_log = """2024-01-15 DENY 203.0.113.5
2024-01-15 ALLOW 192.168.1.10
2024-01-15 DENY 198.51.100.99
2024-01-15 ALLOW 10.0.0.5"""

# Find all DENY lines (lines that start with a date and contain DENY)
# re.MULTILINE makes ^ match at the start of each line
deny_lines = re.findall(r"^.+DENY.+$", firewall_log, re.MULTILINE)
print("DENY lines:")
for line in deny_lines:
    print(" ", line)
```

Practice - Section 5

Write: Validate whether each string in `test_cases` is a correctly formatted **IPv4 address** (four groups of 1–3 digits separated by dots, nothing else before or after). Print `VALID` or `INVALID` for each.

In [ ]:
```
import re

test_cases = [
    "10.0.0.1",
    "192.168.100.254",
    "256.0.0.1",        # 256 is out of range - but note: regex can't check numeric
    "10.0.0",           # only 3 groups
    "10.0.0.1.5",       # 5 groups
    "10.0.0.1 extra",   # trailing characters
    "abc.def.ghi.jkl",  # letters not digits
]

# Your code here
```

▶ 💡 **SOLUTION**

---

6. Groups and Capturing

Analogy: In a structured log, you don't just want to know that a line matches a pattern — you want to **extract the parts**: the timestamp, the source IP, the action, the destination port. Groups are the mechanism for pulling out specific pieces from a match.

Syntax	Meaning
`(pattern)`	Capturing group — extracts the matched text
`(?P<name>pattern)`	Named capturing group — access by name
`(?:pattern)`	Non-capturing group — group without extracting
`\\\|`	Alternation — match either left OR right (`DENY\\\|ALLOW`)

In [ ]:
```
import re

# Numbered groups: group(0) = whole match, group(1) = first (), group(2) = second (
log_line = "2024-01-15T08:30:42Z DENY 203.0.113.5 -> 10.0.0.1 port=22"

pattern = r"(\d{4}-\d{2}-\d{2})T([\d:]+)Z (DENY|ALLOW) ([\d.]+) -> ([\d.]+) port=(\

m = re.search(pattern, log_line)
if m:
    print("Full match:  ", m.group(0))
    print("Date:        ", m.group(1))   # 2024-01-15
    print("Time:        ", m.group(2))   # 08:30:42
    print("Action:      ", m.group(3))   # DENY
```

```
    print("Source IP:   ", m.group(4))   # 203.0.113.5
    print("Dest IP:     ", m.group(5))   # 10.0.0.1
    print("Port:        ", m.group(6))   # 22
```

Named groups — the recommended approach for log parsing

Numbered groups become hard to maintain when the pattern changes. Named groups (`?P<name>`) make patterns self-documenting and robust.

In [ ]:
```
import re

log_line = "2024-01-15T08:30:42Z DENY 203.0.113.5 -> 10.0.0.1 port=22"

# Named groups: access by name instead of number
pattern = (
    r"(?P<date>\d{4}-\d{2}-\d{2})"
    r"T(?P<time>[\d:]+)Z "
    r"(?P<action>DENY|ALLOW) "
    r"(?P<src_ip>[\d.]+) -> "
    r"(?P<dst_ip>[\d.]+) "
    r"port=(?P<port>\d+)"
)

m = re.search(pattern, log_line)
if m:
    print("Action:   ", m.group("action"))
    print("Source:   ", m.group("src_ip"))
    print("Port:     ", m.group("port"))

    # .groupdict() converts all named groups to a dict - perfect for further proces
    fields = m.groupdict()
    print("\nAs dict:", fields)
```

In [ ]:
```
import re

# Parse multiple log lines using the same named-group pattern
ssh_auth_log = [
    "Jan 15 08:01:03 server01 sshd: Failed password for admin from 203.0.113.5 port
    "Jan 15 08:01:09 server01 sshd: Failed password for root from 203.0.113.5 port
    "Jan 15 08:02:00 server01 sshd: Accepted password for j.smith from 192.168.1.10
]

pattern = re.compile(
    r"(?P<result>Failed|Accepted) password for (?P<username>\S+) "
    r"from (?P<src_ip>[\d.]+) port (?P<port>\d+)"
)

for line in ssh_auth_log:
    m = pattern.search(line)
    if m:
        d = m.groupdict()
        status = "ALERT" if d["result"] == "Failed" else "OK"
        print(f"[{status}] {d['result']:8} | user={d['username']:8} | src={d['src_i
```

Non-capturing groups `(?:...)`

Sometimes you need grouping for logic (alternation, quantifiers) but **don't** want to capture the result. Use `(?:...)` to group without creating a capture group.

In [ ]:
```
import re

# Alternation with | requires grouping - but we don't need to capture the protocol
endpoint = "tcp://10.0.0.1:443"

# Capturing group: group(1) = 'tcp', group(2) = '10.0.0.1', group(3) = '443'
m = re.search(r"(tcp|udp)://([\d.]+):(\d+)", endpoint)
if m:
    print("With capturing:", m.groups())     # ('tcp', '10.0.0.1', '443')

# Non-capturing group: group(1) = '10.0.0.1', group(2) = '443'
# We group tcp|udp for the | to work, but we don't need that value
m = re.search(r"(?:tcp|udp)://([\d.]+):(\d+)", endpoint)
if m:
    print("Non-capturing:", m.groups())      # ('10.0.0.1', '443') - protocol exclu
```

Practice - Section 6

Write: Parse all lines in the nginx access log below using a **named-group pattern**. For each line, extract `src_ip`, `method`, `path`, and `status_code`. Print a formatted summary of each request.

In [ ]:
```
import re

nginx_log = [
    '10.0.0.5 - - [15/Jan/2024:08:01:01 +0000] "GET /admin HTTP/1.1" 403 512',
    '10.0.0.5 - - [15/Jan/2024:08:01:02 +0000] "GET /admin/../etc/passwd HTTP/1.1"
    '192.168.1.10 - j.smith [15/Jan/2024:08:01:03 +0000] "POST /login HTTP/1.1" 200
    '10.0.0.5 - - [15/Jan/2024:08:01:04 +0000] "GET /.env HTTP/1.1" 404 128',
]

# Your code here - write a named-group pattern and parse each line
# Expected output (one line per log entry):
# 10.0.0.5        | GET  | /admin              | 403
# ... etc.
```

▶ 💡 **SOLUTION**

---

7. Real-World Security Patterns

This section puts everything together with patterns you will actually use in security work. Each pattern is explained building-block by building-block.

7.1 IPv4 Address Validation

In [ ]:
```
import re

# Full explanation of the IPv4 pattern:
#
# ^               - start of string (nothing before the IP)
# (\d{1,3}\.)     - one group of 1-3 digits followed by a literal dot
# {3}             - repeat that group exactly 3 times (first 3 octets)
# \d{1,3}         - final octet (no trailing dot)
# $               - end of string (nothing after the IP)

ipv4 = re.compile(r"^(\d{1,3}\.){3}\d{1,3}$")

# For full validation, add a range check in Python (regex can't check 0-255)
def is_valid_ipv4(ip: str) -> bool:
    if not ipv4.match(ip):
        return False
    return all(0 <= int(octet) <= 255 for octet in ip.split("."))


test_ips = ["10.0.0.1", "192.168.1.254", "256.0.0.1", "10.0.0", "10.0.0.1.5"]
for ip in test_ips:
    print(f"{ip:20} -> {'VALID' if is_valid_ipv4(ip) else 'INVALID'}")
```

7.2 Email Address Validation

In [ ]:
```
import re

# Email pattern breakdown:
#
# ^                   - start of string
# [a-zA-Z0-9._%+-]+   - local part: alphanumeric + allowed special chars, 1 or more
# @                   - literal @
# [a-zA-Z0-9.-]+      - domain name: alphanumeric, dots, hyphens
# \.                  - literal dot before TLD
# [a-zA-Z]{2,}        - TLD: 2 or more letters (com, org, io, co.uk...)
# $                   - end of string

email_pattern = re.compile(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$")

test_emails = [
    "j.smith@corp.com",
    "admin@internal.corp.local",
```

```
    "not-an-email",
    "missing@tld.",
    "user@.com",
    "phishing@ev1l-domain.ru",   # valid format, suspicious content
]

for email in test_emails:
    result = "VALID FORMAT" if email_pattern.match(email) else "INVALID FORMAT"
    print(f"{email:35} -> {result}")
```

7.3 Password Complexity Validation

In [ ]:
```
import re

# Password requirements (common policy):
# - At least 8 characters
# - At least one uppercase letter
# - At least one lowercase letter
# - At least one digit
# - At least one special character
#
# Strategy: use MULTIPLE lookaheads rather than one complex pattern
# (?=...) is a LOOKAHEAD - it asserts a condition without consuming characters

password_pattern = re.compile(
    r"^"              # start
    r"(?=.*[A-Z])"    # lookahead: at least one uppercase letter somewhere
    r"(?=.*[a-z])"    # lookahead: at least one lowercase letter
    r"(?=.*\d)"       # lookahead: at least one digit
    r"(?=.*[!@#$%^&*()_+\-=\[\]{};':,./<>?])"  # lookahead: at least one special ch
    r".{8,}"          # total: at least 8 characters of any type
    r"$"              # end
)

test_passwords = [
    "hunter2",            # too short, no uppercase, no special
    "Hunter2!",           # valid
    "alllowercase1!",     # missing uppercase
    "ALLUPPERCASE1!",     # missing lowercase
    "NoDigitsHere!",      # missing digit
    "NoSpecial123",       # missing special char
    "C0rrectH0rse#",      # valid
]

for pw in test_passwords:
    result = "PASSES POLICY" if password_pattern.match(pw) else "FAILS POLICY"
    print(f"{pw:20} -> {result}")
```

7.4 Log Line Parsing

In [ ]:
```
import re
from typing import Optional
```

```
# Parse a standard syslog-format firewall log line
# Format: TIMESTAMP HOSTNAME PROCESS: ACTION SRC_IP -> DST_IP:PORT

log_pattern = re.compile(
    r"(?P<timestamp>\w{3} +\d{1,2} [\d:]+) "      # Jan 15 08:30:42
    r"(?P<hostname>\S+) "                           # fw01
    r"(?P<process>\S+): "                           # kernel:
    r"(?P<action>ALLOW|DENY|DROP|REJECT) "          # action
    r"(?P<src>[\d.]+):(?P<src_port>\d+) "          # src IP:port
    r"-> "
    r"(?P<dst>[\d.]+):(?P<dst_port>\d+)"           # dst IP:port
)

sample_logs = [
    "Jan 15 08:30:42 fw01 kernel: DENY 203.0.113.5:4444 -> 10.0.0.1:22",
    "Jan 15 08:30:45 fw01 kernel: ALLOW 192.168.1.10:54321 -> 10.0.0.1:443",
    "Jan 15 08:31:00 fw01 kernel: DROP 198.51.100.9:1337 -> 10.0.0.2:3389",
    "Jan 15 08:31:05 fw01 kernel: ALLOW 10.0.0.5:60001 -> 8.8.8.8:53",
    "not a valid log line - should be skipped",
]

dangerous_dst_ports = {"22", "23", "3389", "445"}

print(f"{'Timestamp':20} {'Action':6} {'Source':22} {'Destination':22} {'Flag'}")
print("-" * 80)
for line in sample_logs:
    m = log_pattern.search(line)
    if not m:
        continue   # skip lines that don't match the expected format
    d = m.groupdict()
    flag = " [SENSITIVE PORT]" if d["dst_port"] in dangerous_dst_ports else ""
    print(
        f"{d['timestamp']:20} "
        f"{d['action']:6} "
        f"{d['src']:15}:{d['src_port']:5} "
        f"{d['dst']:15}:{d['dst_port']:5}"
        f"{flag}"
    )
```

Practice - Section 7

Write: Parse the Windows Event Log lines below. Each line contains an Event ID in the format `EventID: NNNN`. Extract the event ID and timestamp from each line, and use the `EVENT_DESCRIPTIONS` dict to print a human-readable description for each.

In [ ]:
```
import re

EVENT_DESCRIPTIONS = {
    "4624": "Successful logon",
    "4625": "Failed logon",
    "4648": "Logon using explicit credentials",
    "4720": "User account created",
    "4732": "User added to security group",
    "4776": "Credential validation attempt",
```

```
}

event_log = [
    "2024-01-15T08:30:01Z [Security] EventID: 4625 Account: j.smith Source: 10.0.0.
    "2024-01-15T08:30:05Z [Security] EventID: 4625 Account: j.smith Source: 10.0.0.
    "2024-01-15T08:30:11Z [Security] EventID: 4720 Account: backdoor_user Source: 1
    "2024-01-15T08:30:15Z [Security] EventID: 4732 Account: backdoor_user Source: 1
    "2024-01-15T08:31:00Z [Security] EventID: 4624 Account: j.smith Source: 192.168
]

# Your code here - extract timestamp and event_id, print with description
```

▶ 💡 **SOLUTION**

---

8. Summary

The `re` module - four functions

Function	Use when
`re.match(p, s)`	Validating: string must START with the pattern
`re.search(p, s)`	Finding: pattern may appear ANYWHERE in the string
`re.findall(p, s)`	Extracting: get ALL matches as a list
`re.sub(p, r, s)`	Replacing: redact, sanitise, or transform matches
`re.compile(p)`	Reuse: compile once, call `.search()` / `.findall()` many times

Building blocks

Symbol	Meaning
`\d`	Any digit
`\w`	Any word character (letter, digit, underscore)
`\s`	Any whitespace
`.`	Any character except newline
`[abc]`	One of: a, b, or c
`[^abc]`	Anything except a, b, c
`+`	1 or more
`*`	0 or more
`?`	Optional (0 or 1)

Symbol	Meaning
`{n,m}`	Between n and m times
`^`	Start of string
`$`	End of string
`\b`	Word boundary
`(...)`	Capturing group
`(?P<name>...)`	Named capturing group
`(?:...)`	Non-capturing group
`a|b`	a or b
`+?` / `*?`	Lazy (match as little as possible)
`(?=...)`	Lookahead (assert without consuming)

Key rules to remember

1. **Always use raw strings** `r"pattern"` — prevents Python from eating backslashes.
2. **Always check for** `None` before calling `.group()` — `search()` and `match()` return `None` on no match.
3. **Use** `^` **and** `$` **for validation** — without them, partial matches will fool you.
4. **Prefer named groups** `(?P<name>...)` for log parsing — much easier to maintain.
5. **Compile patterns used in loops** — `re.compile()` avoids re-parsing on every iteration.
6. **Regex validates format, not values** — use Python to check numeric ranges (0–255 for IPs, 1–65535 for ports).

Next Steps

- Complete **01_Drills_Regex.ipynb** - 15 exercises covering all pattern types
- Practice on regex101.com — paste any pattern to see it explained step by step with live test strings